TECHNICAL HANDOVER HANDBOOK

SmartReco: End-to-End Design, Implementation & Validation

Behavioral AI recommendations from browser event to semantic retrieval, grounded generation, observability, delivery, and testing

Reference implementation · repository commit 7c0c12fd155d

8 August 2026

Purpose of this handbook

This document is the technical handover for SmartReco. It explains the project as one system rather than as a requirement audit. A new engineer should be able to read it, understand why the architecture exists, locate the implementation, reproduce the application locally, follow a recommendation from the browser to the database and AI services, and verify each important behavior with tests and evaluation commands.

The implementation described here is aligned to repository commit 7c0c12fd155da472a510791e98fda425b3b35349 as of 8 August 2026. Where the older 'How SmartReco Recommends' guide differs from current code, this handbook follows current executable behavior.

CORE DESIGN PRINCIPLE

SmartReco does not ask an LLM to guess which courses exist or to freely choose products. Behavioral evidence and semantic retrieval produce catalog candidates; deterministic eligibility and ranking choose the courses; SQL/MCP verifies them; the Mesh-hosted LLM writes persuasive copy for those exact IDs; validation checks the result before it is stored and shown.

What SmartReco is

SmartReco is a behavioral recommendation platform for a learning marketplace. It observes authenticated learner activity, converts those events into time-decayed interest evidence, retrieves real products semantically from Qdrant, combines semantic relevance with behavioral and learning-path signals, and generates a short personalized explanation through Mesh API. Recommendations are persisted, refreshed as meaningful evidence changes, and can be delivered proactively to learners who explicitly enable digest delivery.

There are two recommendation experiences. The home-page 'Recommended for you' experience is behavior-first and runs only after enough new evidence accumulates. The course-detail 'You may also be interested in' experience is context-first: the current course is a hard relevance anchor and the learner profile changes the order and direction within defensible learning paths.

Architecture at a glance

```
Authenticated browser
  -> batched behavioral events
  -> FastAPI validation + SQL activity_events
  -> deterministic signal derivation + 72-hour decay
  -> user_interest_profiles
  -> trigger / cache / idempotency gate
  -> LangGraph recommendation workflow
```

```
  -> Mesh embedding -> Qdrant semantic retrieval
  -> SQL eligibility + deterministic hybrid ranking
  -> read-only MCP/SQL verification
  -> Mesh persuasive copy
  -> output validation / deterministic fallback
  -> recommendations + recommendation_items
  -> Jinja UI + feedback events
  -> optional APScheduler digest -> sandbox or SMTP
  -> local ServiceInvocation telemetry + optional LangSmith traces
```

SQL is the source of business truth. Qdrant is a rebuildable semantic retrieval index. The browser never authorizes itself by sending a user ID; the server resolves the authenticated user from the session. The LLM does not write business records directly. Background schedulers handle catalog synchronization, stale-run recovery, digest creation/dispatch, retention, and LangSmith reconciliation.

Technology stack and responsibility boundaries

Technology	Primary responsibility	Important boundary
FastAPI	HTTP routes, auth-aware APIs, admin/storefront endpoints, validation and orchestration entry points.	Required backend; business authorization remains server-side.
Jinja2 + JavaScript	Server-rendered learner/admin UI and non-blocking behavioral tracking.	JavaScript records events; it does not become the source of identity or catalog truth.
SQLAlchemy + SQLite/PostgreSQL	Users, sessions, catalog, events, signals, profiles, runs, recommendations, delivery and telemetry.	Authoritative business state.
Qdrant	Semantic product retrieval.	Rebuildable index; every returned ID is re-checked against active SQL state.
Mesh API + OpenAI SDK	Semantic embeddings and recommendation-copy LLM calls.	Every AI/LLM call goes through the configured Mesh base URL.
LangGraph	Explicit recommendation workflow, conditional routing and bounded refinement.	Orchestrator, not an LLM or vector database.
LangSmith	Hosted tracing for graph/retriever/tool/provider-attempt spans.	Optional observability; local SQL telemetry remains authoritative.
APScheduler	Catalog sync, stale-run recovery, digest scheduling/dispatch, retention and trace reconciliation.	Deploy one scheduler leader per deployment.
MCP	Read-only profile/catalog/candidate tools and exact product verification.	Identity-bearing tools are intentionally trusted-local stdio only.

Repository organization

Application entry and routes: app/main.py starts the FastAPI application; app/routes.py contains learner APIs, authentication-facing pages, event ingestion, recommendation endpoints and admin views.

Database model: app/models.py defines durable business, security, delivery and observability entities; Alembic migrations keep schema changes reproducible.

Behavior processing: app/services/signals.py converts activity into signals and maintains the decayed UserInterestProfile.

Recommendation engine: app/services/recommendation.py contains retrieval, contextual ranking, LangGraph state/nodes/edges, trigger gating, cache rules, persistence and stale-run recovery.

AI gateway: app/services/mesh.py owns Mesh embeddings, prompt construction, structured copy parsing, PII minimization and deterministic copy fallback.

Catalog/vector consistency: app/services/catalog.py creates versioned outbox events; app/services/vector_store.py synchronizes/rebuilds/verifies Qdrant and semantic provenance.

Observability: app/services/observability.py stores local invocation evidence; app/services/langsmith_reconciliation.py matches/backfills LangSmith provider spans.

Delivery and retention: app/services/delivery.py owns digest scheduling/dispatch/retry/freshness; app/services/retention.py enforces data-retention windows.

Browser tracker: app/static/js/tracker.js queues, batches, retries and records view/search/click/dwell/cart/recommendation activity without blocking navigation.

Identity, authentication and authorization

SmartReco uses email/password authentication with two application roles: user and admin. Passwords are stored as Argon2 hashes. Login creates an opaque random server-side session; only the session token reaches the browser and its hash is persisted. The session record also contains a CSRF token and expiry/revocation state. Admin pages and catalog mutations are protected by server-side role checks, not by hiding UI elements.

Registration: The registration domain is configurable through REGISTRATION_EMAIL_DOMAIN. Registration creates the learner and the initial personalization profile in the same transaction.

Login abuse: AuthAttempt records plus a configurable attempt/window policy throttle repeated failed logins.

CSRF: State-changing browser requests use a session-bound CSRF token; event ingestion also requires the CSRF header.

Session security: Logout revokes the server-side session. Production configuration requires a non-default APP_SECRET and secure cookies.

Host/browser hardening: Trusted hosts and security headers reduce host-header and browser attack surface; production should run behind HTTPS with COOKIE_SECURE=true.

Object-level authorization: Recommendation feedback validates that the recommendation belongs to the authenticated learner and that the referenced product is actually one of its items.

Core relational data model

The schema separates raw evidence, derived evidence, workflow execution and final recommendations. That separation is important: raw events can be audited, signals can be recomputed, profiles can evolve, and recommendation runs can fail/retry without destroying the original behavior.

Entity	Role in the system
User / UserSession / AuthAttempt	Identity, role, preferences, opaque session state and login-abuse evidence.
Product	Authoritative course/catalog record with status, version and content checksum.
CatalogOutbox / ProductVectorState	Versioned SQL-to-Qdrant synchronization work and verified index provenance/state.
ActivityEvent	Immutable-ish accepted browser behavior: who, what, when and relevant product/search/recommendation context.
BehavioralSignal	Typed evidence derived from one or more events with strength, confidence, topic and expiry.
UserInterestProfile	Current primary/secondary intent, topic weights, recent searches, positive/negative products, journey stage, trigger score and profile hash.
RecommendationRun	One durable orchestration attempt: scope, trigger, profile hash, graph state, node, retrieval metrics, tokens/cost, lease and errors.
Recommendation / RecommendationItem	Stored learner-facing narrative plus ordered catalog items, component scores, explanation and product version.
Delivery / DeliveryAttempt	Scheduled proactive email work, idempotency, retry status and provider evidence.
ServiceInvocation / TraceReconciliationRun	Durable dependency telemetry and local-to-LangSmith reconciliation snapshots.
AuditLog	Administrative action trail.
RecommendationExposure	Exposure-attribution schema retained for future/auxiliary use; current feedback measurement primarily uses ActivityEvent recommendation events.

Admin catalog management and SQL-to-vector synchronization

Admin create, edit and archive actions change SQL first. In the same SQL transaction, app/services/catalog.py writes a CatalogOutbox record containing product ID, event type, product version and canonical payload. The browser request does not attempt a fragile distributed transaction across SQL and Qdrant. Instead, SQL atomically records both the business mutation and the work required to make the vector index converge.

Create: create_product() inserts Product and a product.upsert outbox event for version 1.

Update: update_product() mutates fields, increments version/checksum and emits product.upsert unless the record is archived.

Delete: archive_product() implements recoverable soft deletion, increments version and emits product.delete so archived products disappear from semantic retrieval eligibility.

Worker: sync_pending_catalog() claims pending/failed work with compare-and-set updates and leases. Stale leases can be reclaimed; failures use retry/backoff; an older event is superseded when a newer SQL product version exists.

Final-version-wins: The worker compares outbox product_version with the current Product.version so delayed older mutations cannot overwrite newer catalog truth.

Provenance: ProductVectorState stores embedding provider/model/dimension, index schema version, product version and checksum. Incompatible/stale semantic indexes are detected rather than silently reused.

Semantic index modes

SEMANTIC: Verified embeddings created through Mesh using the configured semantic embedding model; the reproducible semantic evaluation uses openai/text-embedding-3-small at dimension 1536.

DEGRADED: Development/test deterministic hash vectors used only when explicitly allowed. They are useful for local determinism but are never claimed to be semantic AI retrieval.

UNAVAILABLE: Retrieval provider/index failed. The recommendation graph sees this status and applies the retrieval-quality contract rather than pretending semantic search succeeded.

Production startup requires MESH_API_KEY plus MESH_EMBEDDINGS_ENABLED=true. The rebuild command generates the complete new embedding set before replacing the index and verifies the resulting provenance against SQL.

Behavioral event tracking

The browser tracker is deliberately asynchronous. enqueue() creates a UUID event with the local session ID and timestamp, stores it in a user-namespaced localStorage queue, and triggers a flush after 10 queued events. flush() sends up to 100 events per request using fetch(..., keepalive=true). If the request fails, the batch is prepended back to the queue, capped to the most recent 500 events. A 1.5-second periodic flush handles normal low-volume interaction.

Product impressions use IntersectionObserver and are recorded once when at least half of the card becomes visible. Active dwell is not naive wall-clock time: the tracker adds elapsed time only while the document is visible, begins reporting after 15 active seconds, and then sends roughly 15-second

checkpoints. The backend aggregates repeated dwell checkpoints for the same learner/session/course into one HIGH_ENGAGEMENT signal rather than flooding the profile.

Event vocabulary and recommendation influence

Event	Signal strength	Trigger points	Interpretation
page_viewed	0.05	0.5	Weak navigation evidence.
category_selected	0.25	1.0	Explicit topic exploration.
search_submitted	0.70	6.0	Strong stated intent.
product_impression	0.03	0.0	Visible exposure; never triggers a run by itself.
product_viewed	0.35	3.0	Course detail opened.
product_clicked	0.55	5.0	Deliberate course selection.
active_dwell	dynamic	4.0 first / 1.0 update	At least 15 seconds of active visible viewing.
added_to_cart	0.95	10.0	Strong purchase/save intent.
cart_viewed	0.30	1.5	Learner is reconsidering saved content.
removed_from_cart	0.00	4.0	Course becomes eligible again; not a dislike signal.
recommendation_impression	0.00	0.0	Recommendation exposure measurement.
recommendation_clicked	0.75	6.0	Positive recommendation feedback.
recommendation_dismissed	-0.85	6.0	Explicit negative 'Not for me' feedback.
purchase_started	0.90	10.0	High commercial intent.
purchase_completed	1.00	12.0	Conversion and consumed-product evidence.

Server-side event integrity

POST /api/events/batch resolves the current session user, refuses behavioral ingestion when personalization is disabled, validates the declared event type and event-specific required fields, and de-duplicates on event_id. Product references must exist; recommendation feedback must belong to the same user and product item; event properties are allow-listed/bounded; timestamps that are implausibly old or in the future are rejected; the server derives product category from authoritative SQL instead of trusting the browser.

Active dwell must be between 15 seconds and 30 minutes per accepted checkpoint. Search and category events require meaningful values. The backend records occurred_at supplied by the client within the allowed window and its own received_at, giving both event-time and ingestion-time evidence.

From events to behavioral signals

derive_signals() reads only events after UserInterestProfile.source_event_cursor. Each accepted event maps through EVENT_RULES to a signal type, base strength and trigger contribution. The cursor prevents reprocessing old events. Dwell strength increases logarithmically with active seconds and is capped so an extremely long page view cannot dominate the learner profile.

```
dwell_strength = min(0.90, 0.35 + log(1 + min(active_seconds, 300)) / 10)
```

Signals receive confidence = min(1.0, abs(strength) + 0.1) and normally expire after 30 days. Profile rebuilding considers at most the latest 500 non-expired signals. A retention job also deletes expired signals or signals older than the configured signal-retention window.

72-hour recency decay and profile construction

```
topic_weight += signal.strength * signal.confidence * 2 ** (-age_hours / 72)
```

The 72-hour half-life means fresh behavior dominates while older interests fade smoothly: after 72 hours a signal contributes half its original recency weight; after 144 hours one quarter. The strongest positive topic becomes primary_intent, up to three additional positive topics become secondary_intents, and category/topic weights retain the broader ranked picture.

Recent searches: The latest 10 search events are stored in the profile; up to five are used in retrieval/generation context.

Positive products: Products with sufficiently strong positive signals are retained as engagement evidence.

Negative products: A recommendation_dismissed action adds the product to the current negative set. Later positive actions such as recommendation click/cart/purchase clear that negative state.

Journey stage: Derived deterministically as exploration, comparison, purchase_intent or conversion from recent evidence.

Profile hash: SHA-256 over meaningful profile content; used to suppress duplicate recommendation work when the effective profile has not changed.

Trigger score: Separate from topic strength. It answers 'is there enough new evidence to justify another overall workflow?' rather than 'how interested is this learner?'.

When the recommendation agent runs

Overall recommendation trigger

process_activity_and_maybe_recommend() derives signals first. A run is not created when personalization is disabled, no new signals exist, trigger_score is below 8.0, an identical profile already produced a recommendation, or the 20-second cooldown is active. If a run is already queued/running, new profile evidence marks refresh_requested instead of starting a competing run.

Once a run is claimed, trigger_score resets. The idempotency key is derived from user ID, profile hash, catalog revision and prompt version. This prevents the same evidence/catalog/prompt combination from creating duplicate workflows. After a run finishes, at most two automatic catch-up runs can process meaningful evidence that accumulated while it was running; the bound prevents sustained browsing from monopolizing a worker.

Course-detail contextual trigger and cache

Opening a course page queues a course-scoped recommendation without waiting for the overall 8-point threshold. The current course provides strong explicit context. A contextual cache key includes learner profile hash, full catalog revision, current product ID/version and prompt version. Reuse is permitted only when the prior run succeeded, its recommendation is active/unexpired, its recommended items still exist as active products with the same product versions, and the configured contextual TTL (24 hours by default) has not expired.

The page visit owns one contextual workflow. The product_viewed event and dwell generated by that same visit are still recorded into the profile, but process_activity_and_maybe_recommend(..., allow_recommendation=False) reserves them for the next course visit instead of causing a second LLM workflow while the user waits on the current page.

Semantic retrieval and deterministic ranking

Overall retrieval

The overall behavioral query contains the primary learning goal, secondary interests, up to five recent searches and journey stage. Qdrant returns semantic candidates with similarity scores. SQL then loads active products and filters negative/scope exclusions. The ranker scores eligible catalog products using semantic similarity, topic affinity, recent-search vocabulary, catalog rating and popularity; prior positive engagement adds a bounded bonus.

```
overall_score = 0.42 * semantic_similarity
              + 0.25 * behavioral_topic_match
              + 0.18 * recent_search_match
              + 0.08 * catalog_quality
              + 0.07 * popularity
              + 0.08 engagement_bonus  # only for previously positive products
```

Candidates are sorted by final score. Selection initially allows at most two products from the same category so the result has some breadth; if that constraint would leave unused slots, the best remaining eligible products fill them. The graph requests the top three learner-facing products.

Course-detail contextual ranking

Course-detail ranking begins with a hard relationship gate. A candidate must be in the same category as the current course, in a catalog-defined adjacent learning-path category, or share meaningful skills. A globally strong behavioral match alone cannot admit an unrelated course.

Same category: path_relevance = 1.00.

Configured adjacent category: path_relevance = 0.72.

Shared meaningful skill without category edge: path_relevance = 0.58.

No defensible relationship: Candidate is rejected before scoring.

```
context_score = 0.40 * semantic_similarity
              + 0.27 * learning_path_relevance
              + 0.18 * learner_behavior
              + 0.10 * shared_skill_overlap
              + 0.05 * level_progression
```

Learner behavior inside the contextual score is itself bounded: 65% category/topic affinity, 25% recent-search vocabulary and 10% positive-product engagement. Level progression rewards plausible next steps such as beginner to intermediate. The third selected result must add a related-path angle because at most two selected products may share one category.

```
fit_confidence = min(0.98,
    0.35 * semantic + 0.35 * path_relevance + 0.15 * behavior
  + 0.10 * level_progression + 0.05 * evidence_coverage)

interest_likelihood = 1 / (1 + exp(-8 * (context_score - 0.42)))
```

CONFIDENCE INTERPRETATION

confidence_score and interest_likelihood are ranking/fit indicators. They are not statistically calibrated probabilities that a learner will buy, finish or succeed in a course. If the project later has enough real outcomes, calibration should be measured separately.

Eligibility and freshness rules

The current course is excluded from its own contextual results. Carted and purchased courses are excluded because recommending something already saved/consumed wastes inventory. Explicitly dismissed products are excluded until later positive feedback clears the state. Active SQL status is always required. Vector-only IDs cannot bypass SQL. Stored recommendation items carry Product.version so edits/archives invalidate stale display, cache and digest delivery.

LangGraph recommendation workflow

SmartReco constructs StateGraph(RecommendationState) directly with the langgraph package and compiles it with InMemorySaver. The graph is invoked with thread_id equal to RecommendationRun.id. Durable run status, node, lease, retrieval metrics, graph summaries, errors, tokens and outputs live in SQL; the in-memory LangGraph checkpoint is useful during the process but is not the durable business source of truth.

LANGCHAIN RELATIONSHIP

The runtime project does not import or depend on the langchain package. It uses langgraph directly for orchestration and langsmith directly for tracing. LangGraph/LangSmith belong to the broader LangChain ecosystem, but there is no LangChain agent, chain, prompt or retriever framework in SmartReco. LangSmith's run_type='chain' is a trace classification, not use of the LangChain package.

Graph state

RecommendationState carries run_id, user_id, optional context_product_id, profile, query, candidates/selected items, generated copy, model/usage, validation_errors, fallback_used, retrieval_attempt, retrieval_quality and optional refined_query. Nodes return partial state updates; LangGraph merges them through the workflow.

Nodes

Node	Function	Responsibility
load	load_context_node	Load profile, saved/purchased exclusions and optional active current-course context; build initial query.
retrieve	retrieve_node	Run overall or contextual retrieval/ranking, persist metrics and increment retrieval attempt.
evaluate_retrieval	evaluate_retrieval_node	Measure candidate count, best score, semantic status and available behavioral/context evidence.
refine_retrieval	refine_retrieval_node	Build one tighter query from current-course skills/path or explicit primary intent/recent searches.
insufficient_evidence	insufficient_evidence_node	Finish successfully with a no-recommendation outcome after the second weak retrieval; make zero persuasive LLM calls.
verify_with_mcp	verify_with_mcp_node	Re-read selected IDs through the governed active-SQL catalog boundary and require exact ordered preservation.
generate	generate_node	Call Mesh for grounded structured copy; overall may fail over across configured models, contextual uses one bounded attempt.
validate	validate_node	Require exact IDs/no duplicates and reject fabricated price/discount, guarantees, sensitive inferences and instruction-echo patterns.
fallback	fallback_node	Build deterministic safe wording over the same verified products when generated wording fails validation.
persist	persist_node	Supersede old active output in the same scope and store recommendation/items, scores, reasons and product versions.

Edges and bounded agent behavior

```
START -> load -> retrieve -> evaluate_retrieval

quality sufficient -> verify_with_mcp -> generate -> validate
validation valid -> persist -> END
validation invalid -> fallback -> persist -> END

quality insufficient on attempt 1 -> refine_retrieval -> retrieve -> evaluate_retrieval
quality insufficient on attempt 2 -> insufficient_evidence -> END
```

Retrieval quality is sufficient only when candidates exist, the best selected final score is at least 0.05, and semantic retrieval is not UNAVAILABLE unless the workflow still has defensible behavioral/context evidence. The maximum retrieval_attempt is two. This gives the agent an explicit evaluate/refine decision while preventing an open-ended reasoning loop, runaway latency or uncontrolled provider cost.

Run claiming, concurrency and recovery

execute_recommendation_run() claims a queued RecommendationRun with a conditional SQL update and assigns a lease before invoking the graph. Overall leases are 300 seconds; contextual leases are the configured contextual Mesh timeout plus 35 seconds. A uniqueness constraint prevents multiple queued/running runs for the same user/scope. retry_stale_runs() returns expired running work to queued and executes it again. Contextual queueing also detects a queued visit that has stopped progressing for more than 60 seconds.

Mesh API and grounded generation

MeshGateway creates the OpenAI-compatible client with base_url = https://api.meshapi.ai/v1 and MESH_API_KEY. The same gateway owns semantic embeddings and chat completions, ensuring that AI inference remains behind the mandatory Mesh gateway. When no Mesh credentials exist in local development, deterministic embedding/copy fallbacks are visibly non-AI and never presented as successful semantic/provider inference.

Prompt contract

Generation receives a minimized learner representation: primary interest, secondary interests, up to five recent searches, journey stage and optional current-course data. It also receives only the already-selected catalog products with exact IDs and bounded title/category/description/skills/outcomes/fit data. Every learner/search/course field is declared untrusted DATA; the model is explicitly told never to follow instructions found in those fields.

The required response is JSON with a concise headline, two or three warm narrative sentences, and one reason for every exact supplied product ID. The model must not invent products, prices, discounts, outcomes, urgency, personal traits or guarantees. Course-detail copy must explain the connection from the current course to each candidate without claiming the learner completed or mastered the current course merely because it was viewed.

Determinism: temperature = 0 improves repeatability for this structured writing task.

Contextual latency bound: Contextual calls use the configured free model, max 700 output tokens and the shorter contextual timeout.

Overall failover: Overall runs use the configured model_failover_chain (at most three unique models) and stop after success or a non-recoverable gateway/account error.

Provider fallback: If no allowed model succeeds, deterministic_copy() produces grounded wording for the exact selected products so a Mesh outage does not mutate selection truth.

Output validation

Pydantic first validates the RecommendationCopy JSON shape. The LangGraph validate node then compares generated item_copy product IDs with the selected IDs in exact order and rejects duplicates. Generated text is scanned for fabricated price/discount language, guarantees, sensitive-trait claims and prompt/instruction echo. A failing output routes through safe deterministic wording before persistence.

Model strategy

The current configuration separates model tiers from recommendation selection. A free model is used first; openai/gpt-4o-mini is available as a paid comparator and openai/gpt-5.4-mini as a premium comparator. The admin model lab sends the same profile and fixed selected products to explicitly selected models so copy quality can be compared without letting a larger model change which products were recommended.

A paid model should be promoted only when measured structured-output validity, grounding, prohibited-claim rate, persuasive quality, latency, provider reliability and cost justify it. Recommendation-selection confidence is improved primarily through retrieval/ranking/evaluation, not by purchasing a larger copy-writing model.

MCP catalog verification

The project includes a read-only FastMCP server with tools for behavior profile, semantic/ranked catalog search, authoritative product details, recent signals and personalized candidates. The recommendation graph uses get_verified_product_details() as the final catalog verification boundary before generation. Returned IDs must exactly preserve the selected IDs and order or the graph raises instead of generating copy.

Identity-bearing MCP tools intentionally fail closed unless MCP_TRUSTED_LOCAL_ONLY=true. This server is designed for trusted local stdio, not as an unauthenticated multi-tenant network service. There are no MCP mutation tools that can change users, products or recommendations.

Recommendation persistence and display freshness

persist_node() marks the previous active recommendation in the same scope as superseded, then stores Recommendation plus ordered RecommendationItem rows. The parent record includes the profile snapshot, model, generated_at and a seven-day expires_at. Each item stores semantic/behavior/final/confidence scores, reason codes, explanation and the Product.version used during selection.

The home and course-detail loaders do not blindly display the newest row. They require active/non-expired recommendation status and active SQL products, compare the expected item count, and reject items whose stored product_version no longer equals the current Product.version. This prevents catalog edits or archives from leaving stale recommendation cards visible.

Closed-loop feedback

A displayed recommendation is not the end of the system. recommendation_clicked, recommendation_dismissed, cart and purchase events re-enter the same event/signal/profile pipeline. 'Not for me' creates negative product state and removes the exact product from later selection. A later positive action clears that negative state. Carted and purchased products remain excluded from recommendation output while their related topic behavior can still strengthen the learner's broader interest profile.

Scheduled proactive delivery

Digest delivery is a bonus production path built around explicit consent and durable work. Personalization and email consent are separate: a learner may keep personalization enabled while digest_enabled remains false. schedule_due_digests() considers only active users with both personalization and digest enabled, uses the learner's timezone, and schedules at most one daily delivery for the latest active, non-expired overall recommendation.

dispatch_due_deliveries() atomically claims due work, records a DeliveryAttempt, and then re-checks the user, consent flags, recommendation type/status/expiry and every recommendation item's active product version immediately before contacting a provider. A user who opts out after the digest was queued is therefore cancelled before sending. Edited or archived products also cancel stale delivery rather than emailing obsolete recommendations.

Sandbox mode: Default. Produces a realistic receipt without contacting an external mail service; safe for public demos and tests.

SMTP mode: Uses STARTTLS and configured SMTP credentials to send the stored recommendation narrative and item explanations.

Retry: Provider failures use exponential retry up to DELIVERY_MAX_ATTEMPTS; old scheduled work becomes overdue; processing claims abandoned for ten minutes are recoverable.

Scheduler jobs

Job	Schedule	Purpose
catalog-sync	every 20 seconds	Claim/synchronize SQL outbox mutations into Qdrant.
stale-run-recovery	every 1 minute	Requeue recommendation runs whose execution lease expired.
digest-scheduler	every 15 minutes	Create idempotent timezone-aware daily digest work.
delivery-dispatch	every 30 seconds	Claim, revalidate and send due deliveries.
delivery-recovery	every 2 minutes	Recover abandoned delivery processing claims.
retention-enforcement	02:20 UTC daily	Expire/delete old recommendations, events, signals, sessions and auth attempts.
langsmith-reconciliation	default every 30 seconds	Match durable local Mesh attempts with hosted provider spans.

LangSmith observability architecture

LangSmith is optional hosted tracing layered over durable local observability. When LANGSMITH_TRACING=true and LANGSMITH_API_KEY is configured, recommendation.py enables tracing and globally sets LANGSMITH_HIDE_INPUTS=true and LANGSMITH_HIDE_OUTPUTS=true. This is deliberate: LangGraph may emit framework spans automatically, so raw state is hidden globally while custom trace processors expose only a small safe summary.

Trace boundaries

Trace	Run type	Meaning
smartreco-recommendation-run	chain	Top-level recommendation execution.
smartreco-rag-retrieval	retriever	RAG candidate retrieval/ranking boundary.
smartreco-mcp-catalog-verification	tool	Exact-ID active catalog verification.
smartreco-generate-copy	chain	Generation/failover orchestration.
smartreco-mesh-provider-attempt	llm	Exactly one hosted LLM span per real Mesh chat provider attempt.

Custom graph processors export run ID, context product ID, profile version, candidate count, retrieval attempt, selected model, fallback flag and validation-error count rather than raw behavior. Provider trace inputs are replaced by a generic description of the grounded-copy task; outputs expose only generic success/failure wording plus usage metadata.

Pseudonymization and secret minimization

Hosted traces use a pseudonymous learner reference derived from SHA-256(APP_SECRET:user_id) rather than the database user ID. Local SQL keeps the real user relation for authorized admin filtering. sanitize_telemetry() recursively redacts sensitive key names such as password/secret/token/authorization/cookie/CSRF/API-key and secret-looking rsk_, lsv2_ or Bearer values. Mesh-bound free text strips obvious email/phone contact PII, control characters and excessive length while preserving technical vocabulary.

Durable ServiceInvocation ledger

Every recommendation dependency can produce a local ServiceInvocation with service, operation, user/run relation, correlation ID, model, attempt, status, start/completion time, latency, tokens, estimated cost, provider receipt, failover decision, sanitized metadata and error evidence. For a Mesh attempt, link_langsmith_span() saves the exact LangSmith trace/run identity before the provider request begins; successful spans receive token usage and provider receipt/model metadata.

Reconciliation and backfill

reconcile_langsmith_traces() lists local LLM attempts and remote smartreco-mesh-provider-attempt spans from the configured project/window. It matches only the current provider-attempt schema using exact LangSmith run ID or local correlation metadata. Recent missing exports remain pending; older missing exports can be backfilled as explicitly historical spans with original timestamps/status/tokens and hidden payloads. A failed provider attempt remains failed. Reconciliation reports matched, pending, delayed and unmatched counts instead of force-matching data.

This split is intentional: recommendation correctness never depends on LangSmith availability, but a configured environment still gains end-to-end visual traces and an independent usage/latency record that can be compared with local telemetry.

Privacy, safety and data retention

Personalization controls

New learners have personalization enabled by default with an account opt-out. When disabled, behavioral event acceptance, signal/profile updates, contextual/overall graph runs, behavior export to Mesh, digest scheduling and dispatch stop. Existing history is not silently deleted merely by opting out; an authenticated history-deletion action is provided separately.

Digest/email delivery is stricter: digest_enabled defaults false and requires explicit opt-in. Dispatch re-checks that preference immediately before provider contact, so consent withdrawal after queueing is honored.

Retention

Data	Default retention / expiry behavior
Activity events	180 days, then deleted by retention job.
Behavioral signals	Nominal 30-day expires_at and configured 30-day retention; expired signals are excluded from profile rebuild even before deletion.
Recommendations	Active recommendation has seven-day product-facing expiry; non-active historical recommendations are deleted after configured 90-day retention with dependent delivery/items/exposure cleanup.
Sessions	Deleted after session expiry.
Auth attempts	Deleted after configured 30-day retention.
Aggregate profile	Retained as current aggregate state unless the learner uses personalization-history deletion.

Recommendation safety

The safest architectural choice is that the model never selects arbitrary IDs. Qdrant proposes candidates, deterministic code applies eligibility/ranking, SQL/MCP re-verifies them, Mesh explains only those exact products, and post-generation validation requires the same ordered IDs. Prompt injection inside search text/catalog text is treated as data and explicitly ignored. Sensitive personal traits are neither intentionally inferred nor allowed in generated copy.

Deployment boundary

Local development uses SQLite, embedded/local Qdrant and one APScheduler process. Production should use PostgreSQL, remote Qdrant, real Mesh semantic embeddings, a unique strong APP_SECRET, HTTPS/secure cookies, restricted trusted hosts, one scheduler leader, backups and approved external SMTP/LangSmith credentials only when those integrations are enabled. The trusted-local MCP server must not be exposed as a public multi-tenant service without adding principal authentication and authorization.

Local setup and execution

```
Copy-Item .env.example .env
# Configure at least APP_SECRET and DEMO_ADMIN_PASSWORD.
# Add MESH_API_KEY when exercising real Mesh AI/semantic behavior.

python -m pip install -r requirements.txt
python -m alembic upgrade head
python scripts/seed_demo.py --admin-password "choose-a-strong-password"
python -m uvicorn app.main:app --reload
```

The seed creates an admin account and learner demo data/catalog. The application opens at http://127.0.0.1:8000. Use the admin pages for products, activity, runs, deliveries, observability and the model lab; use the learner storefront to create real browser behavior.

Key environment configuration

Setting	Purpose / default behavior
DATABASE_URL	SQLite local default; set PostgreSQL URL for production.
QDRANT_PATH / QDRANT_URL / QDRANT_API_KEY	Local embedded or remote vector store.
MESH_API_KEY / MESH_BASE_URL	Mandatory credential/gateway for real AI calls; base URL is Mesh OpenAI-compatible endpoint.
MESH_EMBEDDINGS_ENABLED	false for explicit degraded local development; true is required in production.
MESH_EMBEDDING_MODEL	openai/text-embedding-3-small by default.
MESH_FREE_MODEL / PAID_MODEL / PREMIUM_MODEL	Copy model tiers used by normal runs/model lab.
RECOMMENDATION_MIN_TRIGGER_SCORE	8.0 default overall refresh threshold.
RECOMMENDATION_COOLDOWN_SECONDS	20-second default duplicate-work cooldown.
CONTEXTUAL_RECOMMENDATION_TTL_HOURS	24-hour contextual cache freshness window.
LANGSMITH_TRACING / API_KEY / PROJECT	Optional hosted trace export; separate from Mesh credentials.
DELIVERY_MODE / SMTP_*	Sandbox by default; configure SMTP for real email.
DIGEST_HOUR_LOCAL	15:00 learner-local target by default.
ACTIVITY/SIGNAL/RECOMMENDATION/AUTH retention settings	UTC retention-policy windows.
MCP_TRUSTED_LOCAL_ONLY	true by default; identity-bearing MCP tools fail closed otherwise.

How to validate the system

Validation is layered. Unit/integration tests prove contracts; deterministic evaluation compares rankers without spending LLM credits; explicit live-Mesh commands verify real semantic provenance and model behavior; manual browser/admin flows demonstrate the experience end to end.

Automated test suite

```
python -m pytest --cov=app --cov-report=term-missing -p no:cacheprovider
```

The final repository validation records 184 passing tests, zero failures and 82% total coverage. Live provider comparisons are opt-in so ordinary CI cannot unexpectedly spend Mesh credits.

Capability-to-test traceability

Capability	Primary automated evidence
Authentication, roles, preferences and learner isolation	tests/test_web.py plus security/schema tests.
Catalog create/update/archive + outbox/vector versioning	tests/test_catalog_and_recommendation.py and tests/test_hardening.py.
Event validation/idempotency and signal derivation	tests/test_security_and_schemas.py, tests/test_signals.py, tests/test_web.py.
Grounded overall/contextual recommendations	tests/test_catalog_and_recommendation.py recommendation pipeline/course-scoped tests.
Context cache invalidation	tests/test_hardening.py::test_context_cache_invalidates_on_profile_course_and_catalog_changes and product-change invalidation test.
Bounded LangGraph retrieval refinement	test_langgraph_records_bounded_retrieval_quality_evidence; test_langgraph_refines_once_then_verifies_and_generates; test_langgraph_stops_after_one_refinement_without_persuasive_llm.
Provider failure fallback	test_provider_connection_failure_uses_grounded_fallback.
Personalization opt-out blocks external AI	test_personalization_opt_out_blocks_signals_context_graph_and_mesh.
Digest consent, idempotency, retry, concurrency and freshness	tests/test_delivery.py including edited/archived product cancellation and two-dispatcher protection.
Telemetry redaction and LangSmith recovery	test_langsmith_and_local_telemetry_redact_credentials; test_langsmith_backfill_replays_missing_attempt_as_historical_span; token reconciliation web test.
MCP trust boundary	test_identity_bearing_mcp_tools_fail_closed_outside_trusted_local_boundary.
Retention	test_retention_enforces_expiry_for_events_signals_sessions_and_auth.

Submission/secret hygiene

```
python scripts/check_submission_hygiene.py
```

The repository .gitignore excludes .env, local databases, caches, logs and Qdrant state. Credentials belong in local .env and GitHub repository secrets, not committed source. The challenge workflow passes SUBMISSION_TOKEN and MESH_API_KEY from GitHub Secrets into the official checker.

Deterministic recommendation evaluation

```
python scripts/evaluate_recommendations.py
python scripts/evaluate_recommendations.py --json
python scripts/evaluate_intent_evolution.py --json
```

The evaluator uses predeclared relevance categories across ten synthetic learner journeys and compares popularity, semantic-only and SmartReco hybrid ranking. It reports Precision@3, Recall@3, NDCG@3, diversity, catalog coverage, personalization separation, exclusion correctness, hallucinated-ID rate, semantic status, latency, embedding calls and copy-LLM calls. Relevance is defined before the run rather than graded by the same LLM being evaluated.

Real semantic index verification

```
python -m alembic upgrade head
python scripts/rebuild_vector_index.py
python scripts/rebuild_vector_index.py --verify-only
python scripts/evaluate_recommendations.py --semantic --json
```

The --semantic evaluator fails closed unless the complete index verifies as real SEMANTIC Mesh/Qdrant state. Final validation measured 50 active SQL products and 50 synchronized vectors with provider mesh_api, model openai/text-embedding-3-small, dimension 1536 and schema smartreco-product-v1.

Measured ranking evidence

System	Precision@3	Recall@3	NDCG@3	Diversity	Coverage	Separation
Popularity	0.1481	0.0424	0.1633	0.9667	0.0800	0.1000
Semantic-only / real Mesh	0.8148	0.2798	0.8038	0.6667	0.4200	0.9467
SmartReco hybrid / real Mesh	0.9630	0.3307	0.9739	0.6667	0.4200	0.9444

The recorded real-semantic run improved NDCG@3 by 21.16% over semantic-only retrieval and 496.39% over popularity. Exclusion correctness was 100% and no hallucinated product ID was observed. Semantic-only and SmartReco each made ten Mesh embedding calls and no copy LLM call because this evaluator isolates retrieval/ranking quality.

AI-call efficiency benchmark

```
python scripts/benchmark_ai_efficiency.py --live-mesh
```

```
100 browser events -> 10 HTTP batches
  -> 96 accepted + 2 rejected + 2 duplicates
  -> 90 signal updates -> 10 trigger evaluations
  -> 2 recommendation workflows -> 2 real Mesh copy calls
  -> 1 contextual cache hit
```

Compared with a deliberately naive baseline of one recommendation-copy call per raw event, this measured run reduced copy-LLM calls by 98%. Batched ingestion was approximately 40 ms median and internal hybrid retrieval/ranking approximately 14.47 ms average in that run; external Mesh copy latency (about 8.6 seconds mean) is reported separately rather than hidden inside internal latency.

End-to-end manual acceptance test

Learner behavior and overall recommendation

Sign in as the demo learner. Search for a topic, open/click relevant courses and allow active dwell or add a course to cart so the overall trigger crosses 8 points. Open Admin > Activity and verify the raw events belong to the learner and that server-side product/category data is present. Open the learner home page after the workflow finishes and verify a stored personalized narrative plus up to three courses appears under Recommended for you.

In Admin > Runs, confirm the run has a behavioral trigger reason, profile hash, retrieval metrics, current/final graph node and selected IDs. In Admin > Observability, confirm RAG/MCP/LangGraph/LLM invocations, provider attempt status/model, tokens/latency and receipt when available.

Course-detail recommendation

Open a course detail page. The contextual lifecycle should queue immediately without waiting for the overall threshold. Inspect the selected recommendations: each must be the same domain, a configured adjacent learning path, or share meaningful skills with the current course. Refreshing unchanged state should reuse the cache rather than fire another Mesh copy call. Change the learner profile or current product/catalog version and verify cache invalidation.

Negative feedback and closed loop

Use 'Not for me' on one recommended course. Confirm recommendation_dismissed appears in Activity and the product enters negative state. Trigger/visit again and confirm the exact dismissed product is

excluded. Perform a later positive action for that product if testing recovery and verify the negative state can clear.

Dual-write and freshness

As admin, create a course and observe the SQL product plus pending outbox state. After catalog-sync, verify ProductVectorState/Qdrant synchronization. Edit the course and confirm version increments and a new upsert event. Archive it and confirm a delete event removes vector eligibility. Any stored recommendation item using an older version must be rejected by display/cache/digest freshness checks.

Privacy and delivery

Disable personalization and verify new behavior is not accepted into the personalization pipeline, contextual/overall graph execution does not start and no behavior reaches Mesh. Re-enable personalization but leave digest disabled; no daily delivery should be created. Enable digest, schedule/dispatch in sandbox and verify receipt/attempt evidence. Disable digest after queueing and verify dispatch cancels before provider contact.

LangSmith demo

With a separate LangSmith API key configured, run a recommendation and inspect the hosted hierarchy: recommendation chain, retriever, MCP tool, generation chain and individual Mesh provider attempt. Confirm hosted inputs/outputs are hidden/minimized and learner identity is pseudonymous. Compare matched token/latency values with the local observability ledger. When LangSmith is absent, verify the same local ServiceInvocation evidence remains available.

Operational failure scenarios

Mesh unavailable: Provider attempt becomes failed; overall failover is bounded by error classification; deterministic grounded copy prevents catalog hallucination when no allowed provider succeeds.

Semantic retrieval unavailable: Retrieval status is UNAVAILABLE; quality evaluation either uses defensible context/behavior evidence or stops after one refinement without persuasive generation.

Stale recommendation run: Lease expiry moves the run back to queued and retry_stale_runs() executes it safely.

Duplicate behavior retry: event_id uniqueness makes repeated browser delivery idempotent.

Two catalog workers: Compare-and-set claims/leases prevent both from independently owning the same outbox record; final-version-wins blocks stale mutation overwrite.

Two delivery workers: Conditional claim from scheduled to processing means only one dispatcher wins; regression tests cover duplicate-send prevention.

Product edited after recommendation: Stored item version becomes stale; display/cache/delivery rejects it instead of showing/sending obsolete data.

LangSmith export delayed: Local attempt remains authoritative; reconciliation reports pending/delayed and can backfill an explicitly historical span later.

User withdraws consent: Ingestion/orchestration/delivery boundaries re-check the personalization/digest flags; queued delivery is cancelled before outbound contact.

Implementation blueprint for a new engineer

If rebuilding SmartReco from an empty repository, implement it in dependency order. Start with identity/catalog and relational schema because every later component depends on authoritative users/products. Add event ingestion before AI. Build signals/profile and deterministic rankers before generation so recommendation selection can be tested without an LLM. Add Qdrant/Mesh semantic retrieval and provenance. Wrap the stable selection pipeline in LangGraph, then add Mesh copy, validation/persistence, UI refresh, delivery and observability. Finish with privacy/retention/concurrency hardening and evaluation scripts.

The most important implementation discipline is to keep inference at the edges of business truth. Do not let the LLM mutate catalog eligibility, authorization, consent, product IDs, prices or persistence logic. Those remain deterministic and testable. This makes the agent explainable, cheaper to run, easier to recover, and safer to deploy.

Source map for maintenance

Concern	Primary implementation
FastAPI app/config/security wiring	app/main.py, app/config.py, app/security.py
Learner/admin HTTP routes	app/routes.py
Relational schema	app/models.py and alembic/
Browser event batching/dwell	app/static/js/tracker.js
Behavioral signals/72-hour profile	app/services/signals.py
Catalog mutation/outbox	app/services/catalog.py
Qdrant sync/provenance/rebuild/search	app/services/vector_store.py
Recommendation ranking/LangGraph/cache	app/services/recommendation.py
Mesh embeddings/copy/fallback/PII minimization	app/services/mesh.py

Concern	Primary implementation
MCP server/catalog tools	app/mcp_server.py, app/services/mcp_catalog.py
Local telemetry	app/services/observability.py
LangSmith reconciliation/backfill	app/services/langsmith_reconciliation.py
Digest delivery/retries/freshness	app/services/delivery.py
Retention	app/services/retention.py
Scheduler	app/scheduler.py
Recommendation evaluation	scripts/evaluate_recommendations.py, scripts/evaluate_intent_evolution.py
AI efficiency benchmark	scripts/benchmark_ai_efficiency.py
Semantic index verification	scripts/rebuild_vector_index.py
Submission/secret hygiene	scripts/check_submission_hygiene.py

Final mental model

ONE-SENTENCE ARCHITECTURE

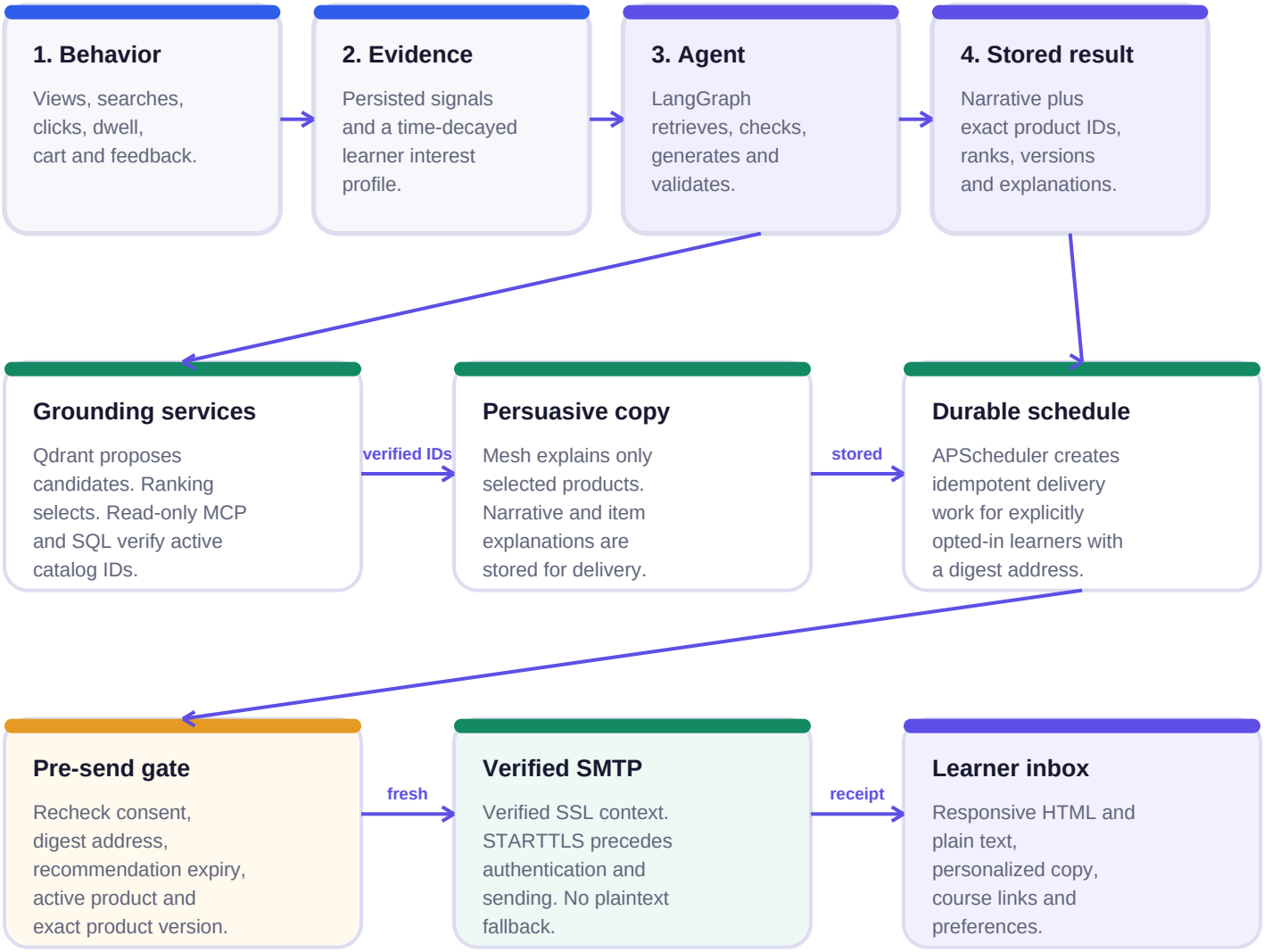
SmartReco observes authenticated behavior, converts it into time-decayed evidence, semantically retrieves real catalog candidates, deterministically selects defensible products, verifies them against SQL, asks Mesh to explain those exact choices persuasively, validates and stores the result, learns from feedback, and proves the lifecycle through durable telemetry, tests and reproducible evaluation.

That separation of concerns is the project's core: behavior determines intent; RAG finds possibilities; deterministic rules protect relevance and business constraints; the LLM communicates value; LangGraph controls the workflow; SQL preserves truth; LangSmith makes the flow observable; APScheduler handles durable background work; tests and evaluation prove that each layer behaves as intended.

SMTP DELIVERY APPENDIX

End-to-end architecture

The email is the final channel in the existing grounded recommendation lifecycle. It does not select courses or bypass consent, SQL freshness, LangGraph, Mesh, Qdrant or the durable delivery ledger.



Failure path - safe, durable and observable

TLS, authentication or provider failure sends no successful receipt. DeliveryAttempt records the sanitized error and applies the existing retry/failure policy. Stale or ineligible recommendations are cancelled before SMTP contact.

STEP-BY-STEP OPERATOR SETUP

Configure an SMTP sandbox

Use an approved sandbox for demonstrations. Mailtrap captures messages without delivering to real learners. Keep every credential in the local environment or deployment secret store.

1

Create sandbox credentials

In Mailtrap, open Email Testing > Sandboxes > your sandbox > Integration > SMTP. Copy host, port, username and password into a secure local secret store.

2

Create a local environment file

Copy .env.example to .env. The repository ignores .env. Never paste credentials into source, README, screenshots, logs or GitHub commits.

3

Set SMTP and public URL

Set DELIVERY_MODE=smtp, SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD, SMTP_FROM and APP_PUBLIC_URL. Port 587 with STARTTLS is the documented default.

4

Enable a controlled learner

Sign in as a test learner, add a Daily digest email address under Account, keep personalization enabled and explicitly opt in to digest delivery.

5

Choose GMT schedule

An administrator sets the daily GMT time from Admin > Delivery. The existing scheduler creates durable, idempotent delivery records for eligible learners.

6

Verify durable evidence

Confirm scheduled -> processing -> sent in the delivery ledger, one attempt, accepted provider status and a sanitized provider receipt. Inspect the message in the sandbox inbox.

```
DELIVERY_MODE=smtp
SMTP_HOST=sandbox.smtp.mailtrap.io
SMTP_PORT=587
SMTP_USERNAME=<provider-username>
SMTP_PASSWORD=<provider-password>
SMTP_FROM=SmartReco <recommendations@smartreco.local>
APP_PUBLIC_URL=http://127.0.0.1:8000
```

WHAT HAPPENS IMMEDIATELY BEFORE SENDING

Delivery lifecycle and guarantees

The dispatcher treats the stored recommendation as a candidate for delivery, not as permanent truth. Every send is guarded by current consent, current catalog state and a verified TLS connection.

01 Claim	A conditional scheduled-to-processing update ensures only one dispatcher wins.
02 Consent	The learner must still be active, personalization-enabled, digest-enabled and have a digest address.
03 Freshness	The recommendation must be active and unexpired. Every item must reference an existing active Product.
04 Version	RecommendationItem.product_version must exactly equal the current Product.version.
05 Compose	The stored headline/narrative and item explanations become multipart plain-text and responsive HTML content.
06 TLS	ssl.create_default_context() enables certificate-chain and hostname verification; STARTTLS precedes authentication.
07 Send	Only after successful TLS does SmartReco authenticate and call send_message.
08 Evidence	Delivery and DeliveryAttempt record status, attempt number, timestamps, provider result and sanitized receipt.
09 Retry	Provider exceptions follow configured exponential retry and final failure behavior. No success receipt is fabricated.

Security invariant

A TLS verification failure cannot reach SMTP authentication, sending, a sent timestamp or a successful provider receipt.

PERSONALIZED, ACTIONABLE AND TESTABLE

Email experience and verification

The presentation layer makes the existing grounded recommendation easier to understand and act on. It does not introduce a second ranker or allow email content to alter selected product IDs.

Personal opening

Learner display name, stored recommendation headline and the behavior-aware narrative generated for that

Course cards

Category, level, exact product title, stored why-it-fits explanation and direct /products/{slug} link.

Clear actions

One button per course, a storefront CTA and an account-preferences link. Plain text carries the same URLs.

Focused verification

- HTML and plain-text MIME alternatives are both present.
- Each active recommendation item has its own product-detail URL.
- Narrative and item explanations come from the stored, validated recommendation.
- STARTTLS receives a default verification-enabled SSLContext.
- Authentication and send happen only after STARTTLS succeeds.
- TLS failure performs no login/send and follows the durable retry path.
- Edited or archived products cancel delivery before provider contact.

Operator commands

```
python -m compileall -q app scripts tests
python -m pytest -q tests/test_delivery.py
python -m pytest -q
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```